AutoBayes brms-parity loop: Tier F review

Status at the Tier F boundary, prepared for Andrew

25 October 2026

This is the Tier F boundary review. Since 21 October you asked for six things: continue with Tier F, benchmark against brms, keep a log of brms's bugs for a later pull request, check that the library reads brms's syntax without extraneous parts, write unit tests for all code before it merges, and move the agent scaffold out of the repository. All of Tier F is on `main`, and every one of the 45 response families brms 2.23 documents now has a verified model. Section 2 is what brms gets wrong, Section 4 answers the syntax question, and Section 8 is what needs you.

1 What was built

Table 1 lists the features. Seven more pull requests changed how the loop works rather than what the library does:

- **Diff coverage (#86).** A feature merges only if every library line it changes runs under the tests. This is how I read your "unit tests for all code before merging". The rule is: 100% of the changed executable lines under `autobayes/`, with exemptions only as a pragma that states its reason. #94 then covered the 13 lines #77 had left unrun.
- **The syntax corpus (#85).** 248 brms formulas, each read by brms and by AutoBayes; a test fails if any reading changes (Section 4).
- **The upstream log (#81).** Every problem in brms, Stan Math, loo and posterior gets a standalone R reproduction. It is kept out of git since #82, as you asked.
- **The benchmark (#84, #104)**: Section 3.
- **The tidy (#100).** The agent scaffold moved to andrew-dou/autobayes-scaffold (private), and the milestone history to `docs/history/`. The top level is now `autobayes`, `docs`, `examples`, `oracle`, `tests`, `tools` and the packaging files.

How close. I measured with the gates' own comparisons on `main` after F08 (`203f5db`). Every gate passed on all 45 Tier F cases, with two exceptions described in Section 6: the two bym2 G1 failures, which are expected, and wiener's expected values, which are accepted against references. The worst relative errors over the Tier F cases:

- 6.7×10^{-16} in G1;
- 1.9×10^{-9} in G2's log density (`cox_uncensored_default_priors`), a fifth of the tolerance. I have not traced its cause.
- 1.2×10^{-8} in G2's gradient (`wiener_distributional_bs`). This is brms's error: Stan's `wiener_lpdf` truncates its series, and AutoBayes's density is exact to 4×10^{-15} against a 60-digit evaluation (D150).
- 6.1×10^{-10} in G3 (`poisson_car_bym2`), which comes through brms's inexact bym2 scaling factor;
- 8.8×10^{-16} in predictions;
- 2.2×10^{-15} between the JAX and PyTensor densities;

Feature	PR	What a user can now write	Diverg.
F01a multinomial	#92	`ab.multinomial("cbind(m1, m2, m3) | trials(size) ~ x + (1 | g)", ...)`, and `ab.dirichlet_multinomial`	D136–D137
F01b simplex	#95	`ab.dirichlet("cbind(s1, s2, s3) ~ x + (1 | g)", ...)`; `ab.logistic_normal(..., refcat="s2")`	D138–D139
F02a CAR	#93	`car(W, gr = site, type = "bym2")` (also escar, esicar, icar), `data2={"W": W}`	D140–D141
F02b SAR	#87	`sar(M, type = "lag")` and `"error"`	D142–D143
F03a fcor	#91	`fcor(V)`, a fixed residual covariance	D144–D145
F03b unstr, cosy	#96	`unstr(time = tt, gr = subj)`, `cosy(time = tt, gr = subj)`	D146–D147
F04 wiener	#88, #90	`ab.wiener("rt | dec(dec) ~ x + (1 | g)", ...)`, with formulas on `bs`	D148–D151
F05 cox	#101, #103	`ab.cox("time | cens(c) + bhaz(df = 7) ~ x + (1 | g)", ...)`	D152–D155
F06a se, rate	#98	`y | se(sdx) ~ 1 + (1 | g)`, `se(sdx, sigma = TRUE)`; `cnt | rate(exposure) ~ x`	D156–D157
F06b thres	#97	`ab.cumulative("ord | thres(gr = z) ~ x + (1 | g)", ...)`, `thres(3, gr = z)`	D158–D159
F06c subset	#99	`["y | subset(sub1) ~ x + (1 | g)", "y2 | subset(sub2) ~ x"]`	D160–D161
F07 summaries	#102	`ab.fixef`, `ranef`, `coef`, `VarCorr`, `summary`, `prior_summary`, `residuals`, `predictive_error`	D162–D165
F08 syntax fixes	#105	ordered factors (`orderedz.L`, `.Q`); `(x | g)`; `gr(g, cor = FALSE)`; `trials(15)`	D168–D169
F09 names, messages	#106	`from autobayes.session import fit_record`: twelve internal names leave the top level; messages without milestone names	D172–D175

Table 1: The Tier F features, verified on 45 frozen brms cases: #83, #89 and #101 froze 38 before the features began, and the features added 7.

- 1.4×10^{-14} in the post-processing tables (`gauss_sar_lag_post`).

Both G2 figures are also the worst over all cases, Tier A to F, outside the cases whose gates are expected to fail. Everything is inside the unchanged tolerances. The full non-sampling suite on `main` after F09 (`5cfcb0b`) passed: 6798 tests passed and none failed, with 193 skipped and 28 expected failures. `oracle/regen.py check -all` regenerated all 216 frozen fixtures from R on `a9b5cf3`, and every one was identical. F08's new case, regenerated on its own afterwards, was identical too.

2 What brms gets wrong

The upstream log now has 52 entries: 41 in brms (one of them not a bug after all), 6 in Stan Math, 3 in loo, and one each in posterior and R. Tier F added 21 of them (U32–U52), each with a script that prints the library's value next to the exact one and exits non-zero while the problem is present. The ones that give a wrong value, or stop where a value exists:

- **SAR models (U32, U33, U36).** A row-standardised weight matrix, the usual kind, cannot be built: R's `eigen()` returns complex eigenvalues and rstan refuses them. `posterior_epred()` of an error-type SAR model stops. Both SAR log densities leave out the Gaussian constant $-\frac{N}{2}\log 2\pi$.
- **The Wiener mean (U34).** `posterior_epred()` uses the relative starting point where the absolute one belongs, so a mean response time can come out negative. AutoBayes's mean is accepted against a 50-digit reference (#90).
- **`bayes_R2()` with `unstr()` or ARMA terms (U41).** When the data are not sorted by group and time, brms pairs the responses with the means in a different order. On one frozen case its value is up to 23.5% off.
- **Cox models (U45–U47).**
  - Interval censoring gives a log density of $-\infty$ at every point.
  - `trunc(lb = L)` normalises each row by its own survival $S(y)$ instead of $S(L)$, which cancels the row's survival term.
  - The default Intercept prior is centred at log(median time), so it moves with the unit of time although the intercept does not.
  - `posterior_linpred()` on new rows stops without the time column, which it does not use (U50).

  AutoBayes refuses the first two, records the third, and predicts without the time column.
- **`factor()` and `ordered()` written in a formula (U51).** New rows that lack one of the levels fail with "non-conformable arguments", and so does a single new row. F08 found it while freezing its ordered-factor case. `lm()` handles the same formula through its recorded levels. AutoBayes keeps the levels it was fitted with (for declared categoricals and for `ordered()`), so new rows need not hold every level.

The rest are inputs brms should refuse and does not:

- disconnected graphs in `car()`, and a diagonal entry counted as an edge (U38, U39);
- SAR bounds of $(\lambda_{\min}, \lambda_{\max})$ where $(1/\lambda_{\min}, 1/\lambda_{\max})$ belongs (U35);
- `dec()` reading any non-zero number as the upper boundary (U40);
- negative multinomial counts (U42), and per-category formulas silently dropping the response formula's terms (U43);
- a zero standard error in `se()` (U44);
- `bhaz(df = 7, knots = c(1.5, 3))`, which silently ignores `df` (U49).

Three are inexact:

- bym2's scaling factor is INLA's perturbed value, not Riebler's exact one: 1.5×10^{-8} to 2.3×10^{-7}

low (U37);
- posterior's `rhat()` and ESS return numbers for chains of two or three iterations (U48);
- R's `contr.poly()`, which brms uses for ordered factors, drifts from the polynomial contrasts as the levels grow: 1.6×10^{-12} at 17 levels, and from 23 levels on its top columns are not polynomial at all (U52). R core answered a related report in 2002 that such polynomials cannot be represented accurately, but the exact ones can be: the script computes them to 10^{-15}. AutoBayes builds them from the definition and refuses more than 16 levels (D168).

Your rule of 24 October settled how AutoBayes treats these: follow brms where it is logical, and otherwise do the sensible thing and record the difference. One case went the other way. For bym2, AutoBayes keeps INLA's definition, computed exactly, because the exact factor would buy nothing practical and would make the model unverifiable against brms (D140).

The drafts in `upstream-drafts/` are ready to post: the maintainer note, the pull-request plan and the log's PDF (Section 8).

3 Speed

The benchmark of 25 October (#104) fitted 18 models with both libraries: the same formulas, priors and data, 4 chains of 1000 warm-up and 1000 kept draws, and target acceptance 0.9.

- **End to end**, compiling and sampling, brms took 0.8 to 18.1 times as long (median 6.5).
- **Its sampler alone** took 0.6 to 13.2 times as long (median 3.8).
- **Where the gain comes from.** The two samplers take about the same number of leapfrog steps. The gain is the gradient: on the median model, B11, one gradient costs 32.8 μs in JAX and 206.9 μs in Stan.
- **Where brms wins.** brms is faster on the horseshoe model (B8) and the exact Gaussian process (B12).
- **Agreement.** The largest difference of posterior means over their combined Monte Carlo error was $|z| = 2.71$ across all 18 models.

4 Does AutoBayes read brms's syntax?

You asked on 24 October whether the code really matches brms's syntax, without extraneous parts. The audit (#85) read 248 formulas with both libraries and compared what each builds, `make_standata()` against AutoBayes's design. The corpus is now a test that fails if any reading changes.

- **Before F08:** 114 of the 116 formulas AutoBayes accepted built brms's data. One differed by design (D9), and one was a silent misreading: an ordered factor was treatment-coded where brms uses polynomial contrasts. Of the 132 refused, 16 were refused without naming the construct.
- **After F08:** 134 formulas match, 113 are refused with the construct named, and the one documented difference remains. Nothing is misread. Ordered factors now use R's polynomial contrasts, built from their definition and checked against brms on a new frozen case. `(x | g)` now means `(1 + x | g)`, as in brms.
- **Extraneous parts (F09).** Twelve names of the scaffold's session records and of the term model's internals (`fit_record`, `environment`, `ModelSpec`, `parse_formula` and others) left `__all__`. Each is still importable from its module, and `ab.<name>` still works with a deprecation warning. `describe()` names the family instead of M0, M1, M2, and about 145 refusals say "not supported" instead of "in this slice" and cite current docs instead of milestone decisions.

No refusal changed what it refuses, and the corpus totals are unchanged. The call-surface differences that had no divergence row now have one: declared factors (D172), how priors cover a class (D173), rows with missing values (D174), and `fit()`'s defaults and the posterior's names (D175). Section 8 lists the four changes F09 proposes but did not make.

What remains refused is listed construct by construct in `docs/current-capabilities.md`. The largest gaps are `s()` and `t2()` smooths, three-way interactions, `poly()`, and many family–context pairs: a family added late often takes the common terms but not `cens()`, `weights()` or a distributional formula.

5 What combining the features found

- **One `data2`.** CAR, SAR and fcor each read a matrix from `data2`, and fcor's and SAR's branches conflicted in 47 places. Their agents unified them into one mechanism used by all three. Since then each agent rebases onto `main` itself and reruns its own checks before reporting, which ended the large conflicts at merge time.
- **Lost lines in rebases.** Resolving conflicts silently dropped three lines: a `continue`, which let `sar()` fall through to the autocorrelation parser, a `@property`, and a function's `return`. The reruns after each rebase caught all three before merge. Agents now check every function both sides touched against `main`.
- **The corpus.** Re-recording all 248 entries rewrote unrelated ones whenever brms's message for some other construct changed. A helper now re-records only the entries a feature changes.
- **A hash-seed-dependent `describe()`.** The order of two lines of one hurdle-cumulative model changes with `PYTHONHASHSEED`. Several agents found it. F09 traced it to Bambi, not brms: `Family.auxiliary_parameters` is a Python set, and `bmb.Model` creates the parameters' variables in its iteration order. It reached past `describe()`: in 25 cases (learned Student-t `nu`, the hurdle and zero-inflated families, wiener and others) the free variables came in a different order in each process. An existing reordering hid it everywhere but that one `describe()`. A seeded fit of those models was then probably not reproducible across processes; this is inferred, since nothing was sampled. The family handed to Bambi now lists them in its likelihood's order, tested in fresh interpreters under two hash seeds.

6 How the claims were checked, and their limits

- **Per feature.** The gate tool ran G1–G6 on the tree that merged. Diff coverage reached 100% on every feature from #87 on, and the corpus test passed. F01a's gate record was lost from the ledger in its rebase; its gate log (GATES GREEN on `f138205`) survives, and the record is restored. Each agent ran the full non-sampling suite on its own branch. I re-ran the focused tests after any rebase.
- **Where brms's number is not the target.** Four Tier F cases are not checked against brms alone:
  - the two bym2 cases, frozen with G1 expected to fail: brms's scaling factor is 1.7×10^{-10} from its own definition, beyond G1's 10^{-12}; every other gate passes (D140);
  - the two wiener cases, whose expected values are accepted against 50-digit references instead of brms's wrong means (D148).
- **Exceptions.** The full suite ran on `main` only at boundaries, not after every merge. Between boundaries a merge rested on the agent's own full suite and my focused re-run.

- **Limits.** Nothing in the loop samples. The only fits are the benchmark's 18 models. No Tier F family has been fitted outside its frozen points, so whether NUTS samples a CAR, SAR or cox posterior well is untested.

7 Process and cost

Agent tokens reported, in millions:

- fixtures: 1.20 (three agents);
- features: F01a 0.94, F01b 0.92, F03b 0.90, F02a 0.84, F04 0.81, F08 0.73, F05 0.72, F06a 0.72, F07 0.67, F06b 0.66, F06c 0.65, F03a 0.63, F02b 0.63, F09 0.53;
- the loop's own tools: upstream log 0.97, syntax audit 0.57, stage benchmark 0.43, tidy 0.27, diff coverage 0.25, #77's lines 0.23;
- about 14.3M in all, against Tier E's 4.6 for six features.

Up to ten agents ran at once. Two things limited the pace: two full suites at a time on this 16 GB machine, and, on 24 October, a disk at 8.9 GB free with swap almost full. Network outages, an expired login and stalled streams stopped agents several times; each resumed from its own context without losing work. I paused three agents once, at 56% of a five-hour window, and resumed them after it reset.

8 Decisions

1. **Posting to brms.** The maintainer note, the pull-request plan and the log's PDF are in `upstream-drafts/`. They go out under your account. I suggest posting the note alone first, as an issue, and writing code only after Paul says how he wants it split.
2. **The scaffold repository** is private. Make it public? Either way, AutoBayes's public history before #100 still contains the scaffold, and #81's history contains the upstream log.
3. **Two withheld results, under your 24 October rule.** Both are places where brms is wrong and AutoBayes computes the right answer, which your rule now covers:
   - the truncated count mean $E[Y \mid Y \geq lb]$, where brms gives $E[Y \mid Y \geq lb + 1]$ (D121);
   - stacking weights at the exact optimum, where loo stops 1.0×10^{-8} short (D126).

   Unless you object, I will release both, each with its divergence row and a 50-digit reference.
4. **Splines.** `s()` and `t2()` are the largest syntax gap left. B04's phase 1 (a basis port checked against R) is on its branch, waiting on the licensing question: GPL'd mgcv, and whether a port written from mgcv's documentation and Wood's papers is acceptable.
5. **CI billing** is still blocking hosted runs; every merge has been verified locally.
6. **Four call-surface changes F09 proposes.** Each departs from today's refusal toward brms; none is made yet.
   - Accept an undeclared pandas `Categorical`, its first category the reference, as R does for a factor; and accept a declared boolean column, which is refused today although brms codes it `FALSE`/`TRUE` (D172).
   - Allow a prior on a whole class plus overrides for single coefficients, brms's `coef` rows, in place of today's mapping that must name every term (D173).
   - An opt-in to drop rows with missing values and report them, as zero-trial rows already are (D174).
   - Change `target_accept`'s default from 0.9 to brms's 0.8 (D175).

   I suggest the first three, under your rule, and keeping 0.9: it costs a little speed and gives fewer divergent transitions, and anyone matching brms can pass 0.8.

7. **Tier G.** Candidates, in the order I would take them:
   (a) the common syntax still refused: three-way interactions, interactions with transforms, and `poly()`, which is built like the ordered-factor contrasts;
   (b) the family–context matrix: `cens()`, `weights()`, distributional formulas and group-level terms for the families that do not yet take them;
   (c) the two releases in item 3, and whichever changes in item 6 you accept;
   (d) a first real fit of each Tier F family, on the GPU or the cluster, which needs your approval for compute;
   (e) splines, once item 4 is decided.

   At Tier F's rate, about 0.75M tokens a feature, (a) and (b) together are perhaps 8–12M.